

Informe Ejecutivo — Evaluación Parcial 3

(Estado de Avance N°3)

Asignatura: TPY1101 — Taller Aplicado de Programación · Sección 801D **Institución:** Duoc UC · Escuela de Informática y Telecomunicaciones · 2026-1 **Docente:** Diego Cares González **Proyecto:** Finanzas Mi Patito — Gestor de finanzas personales con regla 50/30/20 **Repositorio:** <https://github.com/Folaquito/finanzas-mi-patito>
Resultado de Aprendizaje evaluado: RA3 — *Validación y aseguramiento de la calidad del producto*

Integrante	Usuario GitHub	Rol	Responsabilidad principal
Agustín Bahamondes	@AbsolucionArtistica	Product Owner / Analista	Requisitos, modelo de datos, reglas de negocio
Joaquín Fernández	@Folaquito	Desarrollador Frontend	SPA React, consumo de APIs, UX
Diego Bahamondez	@Eth3rn4l	Desarrollador Backend	Spring Boot, persistencia, seguridad

1. Resumen ejecutivo

Finanzas Mi Patito es una aplicación web que permite a una persona registrar sus ingresos y gastos, organizarlos según la metodología presupuestaria **50/30/20** (necesidades / deseos / ahorro) y visualizar su salud financiera. Esta tercera experiencia (RA3) tuvo como propósito **validar que el producto resuelve la necesidad de origen** y que cumple con estándares de calidad y seguridad, mediante la definición y ejecución de un plan de pruebas, la corrección de los defectos detectados y la consolidación de la documentación del proyecto.

Durante esta etapa se incorporó una **batería de pruebas automatizadas (26 casos)** sobre el backend, ejecutadas con éxito (0 fallos), se **corrigieron tres defectos críticos** detectados por el proceso de validación y se realizó una **auditoría**

de seguridad que identificó hallazgos pendientes con su correspondiente plan de remediación. El producto se encuentra **funcional de extremo a extremo** (backend + frontend), con las reglas de negocio implementadas y operativas.

2. Descripción del proyecto

2.1 Necesidad que da origen al desarrollo

Muchas personas no llevan control de en qué gastan su dinero ni cuánto destinan al ahorro. La regla **50/30/20** es un método simple y reconocido para distribuir el ingreso: 50% a necesidades, 30% a deseos y 20% a ahorro. El proyecto operacionaliza esa regla en una herramienta concreta para el contexto chileno (montos en **pesos chilenos, sin decimales**).

2.2 Alcance funcional

- Gestión de usuarios (registro, consulta, actualización, eliminación).
- Autenticación: inicio de sesión, recuperación y cambio de contraseña.
- Gestión de cuentas y de transacciones (ingresos/gastos) con actualización automática de saldo.
- Catálogo de categorías clasificadas según la regla 50/30/20.
- Cálculo del **presupuesto 50/30/20** y de un **resumen financiero** por usuario.
- Gestión de metas de ahorro.
- Interfaz web (SPA) que consume la API: login, dashboard, presupuesto y perfil.

2.3 Objetivos de la etapa RA3

Indicador	Objetivo
IL3.1	Aplicar pruebas de validación a los componentes del proyecto (funcionalidad, buenas prácticas, calidad, seguridad).
IL3.2	Realizar mejoras al producto según el resultado de las pruebas aplicadas.
IL3.3	Elaborar el presente informe con la información relevante del proyecto y sus evidencias.

3. Metodología de desarrollo

El equipo trabajó con un enfoque **ágil iterativo e incremental**, organizado en las tres experiencias de aprendizaje de la asignatura:

1. **EA1 — Determinación y planificación:** definición de alcance, roles, tecnologías y modelo de datos.
2. **EA2 — Diseño y desarrollo:** implementación del backend (CRUD, reglas de negocio) y del frontend.
3. **EA3 — Validación y aseguramiento de calidad:** plan de pruebas, corrección de defectos y documentación final (esta etapa).

Prácticas aplicadas:

- **Control de versiones con Git/GitHub**, con *Conventional Commits* en español (feat:, fix:, docs:, test:, chore:) y flujo de ramas feature/*, fix/*, docs/.*
- **Revisión por responsabilidad:** backend (Diego), frontend (Joaquín), modelo/reglas (Agustín).
- **Documentación viva** del estado del proyecto en docs/progress/ (STATUS.md, DECISIONS.md, CHANGELOG.md), incluyendo registros de decisiones arquitectónicas (ADR).

Referencias metodológicas: *Guía PMBOK* (PMI, 7.^a ed.), Scrum Guide y prácticas de desarrollo orientado por pruebas.

4. Arquitectura de la solución

La solución es un **monolito modular** con separación cliente/servidor:

```
finanzas-mi-patito/
├─ backend/      Spring Boot (API REST)
│  └─ src/main/java/com/finanzas/
│     ├─ config/      SecurityConfig, DataLoader
│     ├─ controller/  Endpoints REST (@RestController)
│     ├─ service/      Lógica de negocio (interfaz + impl)
│     ├─ repository/   Acceso a datos (Spring Data JPA)
│     ├─ model/        Entidades JPA y enums de dominio
│     ├─ dto/          Objetos de transferencia
│     └─ exception/    Manejo global de errores
```

```
└─ frontend/      React + Vite (SPA)
   └─ src/        api/ · context/ · features/ · components/
```

El backend sigue una **arquitectura por capas** (controlador → servicio → repositorio), con inyección de dependencias por constructor y manejo centralizado de excepciones mediante `@RestControllerAdvice`. La comunicación es vía **API REST con JSON**, bajo el prefijo `/api`.

Nota de diseño (ADR-003): la asignatura plantea una arquitectura de microservicios como horizonte. Para esta etapa se optó por un **monolito modular** organizado por capas, decisión documentada en `docs/progress/DECISIONS.md` por su menor riesgo y su idoneidad para el alcance actual, manteniendo una estructura que facilita una futura separación en *bounded contexts* (auth, transacciones, presupuesto).

5. Stack tecnológico

Capa	Tecnología	Versión
Lenguaje backend	Java	21 (LTS)
Framework backend	Spring Boot	3.5.14
Persistencia	Spring Data JPA / Hibernate	—
Base de datos (dev/test)	H2 (in-memory)	—
Seguridad	Spring Security + BCrypt	—
Build backend	Maven	3.x
Frontend	React + Vite	React 19 / Vite
Cliente HTTP	Fetch API (proxy Vite → /api)	—
Pruebas	JUnit 5, Mockito, Spring Boot Test (MockMvc),	—

Capa	Tecnología	Versión
	AssertJ	
Control de versiones	Git / GitHub	—

6. Modelo de datos

6.1 Entidades

Entidad	Atributos clave	Relaciones
Usuario	id, nombre, email (único), password (BCrypt), teléfono, fechaCreación	1:N Cuenta · 1:N Meta
Cuenta	id, nombre, tipo, saldo (BigDecimal)	N:1 Usuario · 1:N Transacción
Categoría	id, nombre, tipo (TipoCategoria)	catálogo
Transacción	id, monto, tipo, tipoMovimiento, descripción, fecha	N:1 Cuenta · N:1 Categoría
Meta	id, nombre, montoObjetivo, montoActual, fechaLímite	N:1 Usuario

6.2 Enumeraciones de dominio

- **TipoCategoria:** NECESIDAD, DESEO, AHORRO — base de la regla 50/30/20.
- **TipoTransaccion:** INGRESO, GASTO.
- **TipoMovimiento:** TRANSFERENCIA, PAGO, DEPOSITO.

6.3 Diccionario de datos (extracto)

Campo	Tipo	Regla
Usuario.email	String	Único, formato de correo válido, obligatorio

Campo	Tipo	Regla
Usuario.password	String	Almacenado con hash BCrypt (nunca en texto plano)
Cuenta.saldo	BigDecimal	CLP entero; se ajusta con cada transacción
Transaccion.monto	BigDecimal	CLP entero, mayor que cero

Todo valor monetario se modela con `BigDecimal` y se expresa en **pesos chilenos sin decimales**.

7. Reglas de negocio

1. **Distribución 50/30/20.** Sobre el total de ingresos del usuario se calcula: $NECESIDAD = 50\%$, $DESEO = 30\%$, $AHORRO = 20\%$. El monto de ahorro se obtiene como $ingreso - necesidad - deseo$, garantizando que **las tres líneas sumen exactamente el ingreso** (redondeo a CLP entero, `HALF_UP`). Por cada categoría se reporta lo presupuestado, lo gastado y el disponible.
2. **Actualización de saldo.** Un `INGRESO` suma al saldo de la cuenta y un `GASTO` lo resta. Al **modificar** una transacción, el sistema revierte el efecto del monto anterior y aplica el nuevo, manteniendo la consistencia del saldo.
3. **Resumen financiero.** Para un usuario se agregan los ingresos y gastos de todas sus cuentas, entregando `totalIngresos`, `totalGastos` y balance.
4. **Seguridad de credenciales.** Las contraseñas se almacenan con **BCrypt**. La recuperación de clave genera un **token de un solo uso con expiración (30 min)**.

7.1 Endpoints principales

Recurso	Operaciones
/api/usuarios	CRUD de usuarios
/api/auth	login, recuperar, reset-clave, cambiar-password

Recurso	Operaciones
/api/cuentas	CRUD + consulta por usuario
/api/transacciones	CRUD + consulta por cuenta (ajuste automático de saldo)
/api/categorias	CRUD del catálogo
/api/metast	CRUD de metas por usuario
/api/presupuesto/{usuarioId}	Cálculo de la distribución 50/30/20
/api/resumen/{usuarioId}	Resumen de ingresos, gastos y balance

8. Ambiente de pruebas

Las pruebas se ejecutan en un **ambiente aislado y reproducible**, independiente del de desarrollo:

- **Perfil de pruebas** con configuración propia (`application-test.properties`).
- **Base de datos H2 in-memory** dedicada (`jdbc:h2:mem:testdb`) con estrategia `create-drop`: el esquema se crea al inicio y se elimina al final de cada ejecución, garantizando independencia entre corridas.
- **Puerto aleatorio** (`server.port=0`) para evitar colisiones.
- **Consola H2 deshabilitada** en pruebas.

Las pruebas de integración levantan el **contexto completo de Spring Boot** y ejercitan los endpoints con **MockMvc**, validando el comportamiento real de la capa web sobre la base de datos de prueba.

Brecha conocida (ver §11): el ambiente de pruebas usa H2, que no es idéntico a un motor productivo (p. ej. PostgreSQL). Se documenta el plan para alinear el ambiente de pruebas con producción.

9. Plan de pruebas y resultados de ejecución (IL3.1)

9.1 Estrategia

El plan de pruebas se estructura a partir de **criterios de aceptación** y **criterios de rendimiento** documentados en `backend/src/test/java/com/finanzas/performance/AcceptanceCriteria.java`, que actúan como contrato verificable. Cada criterio se traduce en uno o más casos de prueba automatizados.

Se aplicaron **cuatro niveles de prueba**:

Nivel	Técnica / Herramienta	Qué valida
Unitaria de servicio	JUnit 5 + Mockito (dependencias simuladas)	Lógica de negocio aislada
Unitaria de controlador	JUnit 5 + Mockito	Delegación y manejo de respuestas/errores HTTP
Integración	Spring Boot Test + MockMvc + H2	Flujo completo controlador→servicio→repositorio→BD
Carga / rendimiento	MockMvc + concurrencia (<code>ExecutorService</code>)	Comportamiento bajo 100 peticiones concurrentes y tiempos de respuesta

9.2 Casos de prueba ejecutados y resultados

Ejecución con `mvn test` (Maven, Java 21). **Resultado global:** `BUILD SUCCESS`.

Suite de pruebas	Tipo	Casos	Fallos	Errores
<code>UsuarioServiceTest</code>	Unitaria de servicio	9	0	0
<code>UsuarioControllerMockTest</code>	Unitaria de controlador	6	0	0
<code>UsuarioIntegrationTest</code>	Integración (MockMvc)	8	0	0
<code>LoadTest</code>	Carga / rendimiento	3	0	0

Suite de pruebas	Tipo	Casos	Fallos	Errores
TOTAL		26	0	0

Cobertura de escenarios (muestra):

- Creación de usuario con datos válidos → 201 Created.
- **Encriptación BCrypt** de la contraseña antes de persistir (CA-002).
- Rechazo de datos inválidos → 400 Bad Request (CA-013).
- Consulta de usuario inexistente → 404 Not Found (CA-012).
- Eliminación → 204 No Content (CA-011).
- **Carga:** $\geq 90\%$ de éxito con 100 usuarios concurrentes (CR-002); tiempos de respuesta dentro de los umbrales definidos (CR-001/CR-004).

9.3 Cobertura y alcance

La batería automatizada cubre de forma **exhaustiva el dominio de Usuario** (servicio, controlador, integración y carga). Los criterios de aceptación correspondientes a **Cuenta, Transacción, Categoría, Presupuesto 50/30/20, Resumen y Autenticación** están **definidos y documentados** en `AcceptanceCriteria.java`, y fueron **verificados funcionalmente de forma manual** durante el desarrollo (pruebas con la API y con la interfaz). Su **automatización** se encuentra planificada como continuación inmediata del plan de pruebas (ver §11).

10. Mejoras realizadas según el resultado de las pruebas (IL3.2)

El proceso de validación detectó defectos que fueron **corregidos durante esta etapa**:

#	Hallazgo detectado	Impacto	Corrección aplicada
1	El plugin de compilación sobrescribía la configuración del <i>parent</i> y omitía la bandera <code>-parameters</code> .	Crítico: todos los endpoints con <code>@PathVariable</code> respondían HTTP 500 al compilar con Maven.	Se añadió <code><parameters>true</parameters></code> al <code>maven-compiler-plugin</code> .
2	Jackson no podía serializar el <i>proxy</i> perezoso de <code>Categoria</code> en los listados de transacciones.	Alto: fallaban los "movimientos recientes" del dashboard.	Se añadió <code>@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})</code> a la entidad.
3	Dependencia <code>mockito-kotlin</code> declarada e innecesaria (proyecto Java).	Medio: rompía la construcción reproducible del proyecto.	Se eliminó la dependencia del <code>pom.xml</code> ; la suite compila y ejecuta limpia.

Adicionalmente, como parte del aseguramiento de calidad:

- Se incorporó un **perfil de pruebas aislado** (`application-test.properties`) para ejecución reproducible.
- Se mantiene el almacenamiento de contraseñas con **BCrypt**.
- Se actualizó la **documentación viva** del proyecto (`docs/progress/`).

11. Auditoría de seguridad y plan de remediación

Como parte de IL3.1/IL3.2 se realizó una revisión de seguridad. Se reportan los hallazgos con su criticidad y plan de corrección, de forma transparente:

#	Hallazgo	Criticidad	Plan de remediación
S1	El campo <code>password</code> (hash) se incluye en las respuestas JSON de los endpoints de usuario, al exponerse la entidad en lugar	Alta	Anotar <code>password</code> con <code>@JsonIgnore</code> y/o exponer un <code>UsuarioResponseDTO</code> sin

#	Hallazgo	Criticidad	Plan de remediación
	de un DTO.		credenciales en todas las respuestas.
S2	<code>SecurityConfig</code> permite todas las solicitudes (<code>permitAll</code>) sin autenticación efectiva ni verificación de propiedad de los datos.	Alta	Activar autenticación, validar <i>ownership</i> por usuario en los endpoints y, a futuro, incorporar JWT (criterios CA-051/CA-052 ya documentados).
S3	La cobertura automatizada se concentra en el dominio Usuario.	Media	Automatizar los criterios de Cuenta, Transacción, Presupuesto 50/30/20 (suma exacta al ingreso) y Resumen.
S4	No se mide la cobertura de código de forma cuantitativa.	Media	Integrar JaCoCo y establecer umbral mínimo de cobertura.
S5	El ambiente de pruebas (H2) no replica un motor productivo.	Media	Definir perfil <code>prod</code> con PostgreSQL y migraciones (Flyway/Liquibase), alineando pruebas con producción.

La detección temprana y documentada de estos hallazgos forma parte del propio proceso de aseguramiento de calidad: están identificados, priorizados y con un plan de acción claro para la siguiente iteración.

12. Evidencias

- **Repositorio:** <https://github.com/Folaquito/finanzas-mi-patito>
- **Capturas de la aplicación funcionando:** `docs/img/login.png`, `docs/img/dashboard.png`, `docs/img/presupuesto.png`.
- **Suite de pruebas:** `backend/src/test/java/com/finanzas/` (26 casos).
- **Criterios de aceptación y rendimiento:** `backend/src/test/java/com/finanzas/performance/AcceptanceCriteria.java`.
- **Salida de ejecución:** `mvn test` → Tests run: 26, Failures: 0, Errors: 0 · BUILD SUCCESS.
- **Documentación de proceso y decisiones:** `docs/progress/STATUS.md`, `docs/progress/DECISIONS.md`, `docs/progress/CHANGELOG.md`.

13. Conclusiones y trabajo futuro

El producto **cumple su propósito**: permite registrar movimientos, aplica la regla **50/30/20** y entrega una visión clara de la situación financiera del usuario, con backend y frontend funcionando de extremo a extremo. El proceso de validación de RA3 permitió **detectar y corregir defectos críticos** que impedían el correcto funcionamiento de la API, y la batería de **26 pruebas automatizadas se ejecuta sin fallos**, dando respaldo objetivo a la calidad del componente de gestión de usuarios.

La auditoría de seguridad y de cobertura entrega una **hoja de ruta priorizada** para elevar el producto a estándar productivo:

1. Corregir la exposición de credenciales (S1) y reforzar la autenticación/autorización (S2).
2. Automatizar las pruebas de las reglas financieras (S3) e integrar medición de cobertura (S4).
3. Alinear el ambiente de pruebas con un motor de producción (S5).

El equipo considera que estas acciones, ya identificadas y planificadas, constituyen la continuación natural del aseguramiento de calidad iniciado en esta etapa.